

CSS Basics

A fully-explained, read-and-teach guide for VS Code

1. What is CSS & Setup in VS Code
2. Selectors (element, class, id)
3. Colors, Text & Fonts
4. The Box Model
5. Backgrounds & Borders
6. Display Property
7. Flexbox Basics
8. Positioning
9. Mini Practice Project

Definitions + Code + Live-teaching lines

1. What is CSS & Setup in VS Code

Definition — CSS: CSS stands for **Cascading Style Sheets**. HTML gives a webpage its structure (headings, paragraphs, buttons), and CSS gives it its look — colors, spacing, fonts, layout. Without CSS, every webpage would just look like plain black text on white background.

Say to class: "CSS never works alone — it needs an HTML file to style. Let's create both files first."

Step 1: Create the files

In VS Code, create a folder `css-class` and inside it two files:

- `index.html` — the structure/content file
- `style.css` — the styling file

```
index.html<!DOCTYPE html>
<html>
<head>
  <title>My CSS Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hello CSS</h1>
  <p>This is a paragraph.</p>
</body>
</html>
```

Definition — the <link> tag: This line `<link rel="stylesheet" href="style.css">` is what connects your HTML file to your CSS file. `rel="stylesheet"` tells the browser "this linked file is a stylesheet", and `href="style.css"` tells it the filename/path to find it. This must go inside `<head>` .

```
style.cssbody {
  background-color: lightyellow;
}
```

Definition — CSS Rule/Syntax: Every CSS instruction is called a "rule" and has 3 parts:

- Selector** — WHAT to style (e.g. `body`)
- Property** — WHICH aspect to change (e.g. `background-color`)
- Value** — WHAT to set it to (e.g. `lightyellow`)

Every property:value pair must end with a semicolon `;` and the whole rule is wrapped in curly braces `{ }` .

Open `index.html` with the "Live Server" VS Code extension (right-click → Open with Live Server) so changes reflect instantly in the browser as you type.

Step 2: The 3 ways to write CSS

Type	What it means	Where it's written
Inline	CSS written directly on one single HTML element	<code>style="color:red;"</code> inside a tag
Internal	CSS written inside the same HTML file, in a style block	Inside <code><style></code> tag in <code><head></code>
External	CSS written in a separate .css file and linked	Separate <code>.css</code> file — this is the best practice, always prefer this

```
<!-- inline example -->
<p style="color:red;">Red text</p>

/* internal example, in <head> */
<style>
  p { color: blue; }
</style>
```

2. Selectors

Definition — Selector: A selector is the part of a CSS rule that tells the browser WHICH HTML element(s) to apply the style to. Think of it as "pointing at" something on the page.

Say to class: "Let's cover the 3 most common ways to point at elements."

1 Element Selector

Definition: Selects ALL elements of that tag type on the page. Just write the tag name with no dot or hash.

```
p {  
  color: darkblue;  
  font-size: 16px;  
}
```

This styles **every** `<p>` tag on the page the same way.

2 Class Selector .

Definition: A "class" is a label you give an HTML element using `class="name"`. In CSS you target it with a **dot** before the name: `.name`. Classes are **reusable** — many different elements can share the same class and all get the same style.

```
<!-- HTML -->  
<p class="highlight">Important text</p>  
  
/* CSS */  
.highlight {  
  background-color: yellow;  
  font-weight: bold;  
}
```

3 ID Selector

Definition: An "id" is also a label, given using `id="name"`, but it must be **unique** — used only ONCE per page. In CSS you target it with a **hash symbol** before the name: `#name`.

```
<!-- HTML -->  
<div id="main-header">Welcome</div>  
  
/* CSS */  
#main-header {  
  text-align: center;  
  color: purple;  
}
```

Quick rule to say out loud: "Dot for class (repeatable), hash for id (one-time-only)."

3. Colors, Text & Fonts

Live demo idea: Change these values live and let students see the browser update instantly.

```
h1 {
  color: #333333;
  font-family: Arial, sans-serif;
  font-size: 32px;
  font-weight: bold;
  text-align: center;
  text-decoration: underline;
  letter-spacing: 2px;
}
```

Property	Definition — what it controls
color	The text (font) color
font-family	Which typeface/font is used to display text
font-size	How big the text is
font-weight	How thick/bold the text looks
text-align	Horizontal alignment of text: left, center, right
text-decoration	Adds/removes underline, strikethrough, etc.
letter-spacing	Space between individual letters

Ways to write colors

Definition: A color value can be written in different formats — they all mean the same thing to the browser, just different notations.

Format	Meaning	Example
Name	Plain English color name	color: red;
Hex	6-digit code, each pair = Red/Green/Blue amount (00-ff)	color: #ff0000;
RGB	Red, Green, Blue values from 0-255	color: rgb(255,0,0);
RGBA	Same as RGB + Alpha (transparency, 0=invisible, 1=solid)	color: rgba(255,0,0,0.5);

4. The Box Model

Definition — Box Model: Every single HTML element on a webpage is treated by the browser as a rectangular BOX. This box has 4 layers, from inside out:

- 1. **Content** — the actual text/image inside
- 2. **Padding** — clear space between the content and the border (inside the box)
- 3. **Border** — the line that wraps around the padding
- 4. **Margin** — clear space OUTSIDE the border, separating this box from other elements

Say to class: "This is the MOST important CSS concept — draw it on the board too: Content → Padding → Border → Margin, like layers of an onion."

```
.box {
  width: 200px;
  padding: 20px;
  border: 3px solid black;
  margin: 15px;
}
```

Property	Definition
width	Sets the width of the content area
padding	Space INSIDE the box, between content and border
border	A visible line around the box — shorthand for width, style, color (e.g. 3px solid black)
margin	Space OUTSIDE the box, pushing other elements away

Shorthand order — TOP RIGHT BOTTOM LEFT (clockwise)

Definition — Shorthand: Instead of writing 4 separate lines for each side, you can write all 4 values in ONE line. The browser reads them clockwise starting from the top.

```
margin: 10px 20px 10px 20px; /* top right bottom left */
margin: 10px 20px;          /* top-bottom | left-right */
margin: 10px;               /* all 4 sides equally */
```

box-sizing trick — always mention this, students get confused why width overflows:

Definition — box-sizing: By default, when you set `width: 200px` , the browser ADDS padding and border on top of that, making the box bigger than 200px. Setting `box-sizing: border-box` forces padding and border to be INCLUDED inside the declared width instead.

```
* {
  box-sizing: border-box;
}
```

5. Backgrounds & Borders

```
.card {
  background-color: #f0f0f0;
  border: 2px solid #999;
  border-radius: 10px;
  box-shadow: 2px 2px 8px rgba(0,0,0,0.2);
}
```

Property	Definition
background-color	Fills the box's background with a solid color
border-radius	Rounds the corners of the box; higher value = more rounded (50% = full circle)
box-shadow	Adds a shadow behind the box — values: horizontal offset, vertical offset, blur, color

Background image

```
.hero {
  background-image: url('bg.jpg');
  background-size: cover;
  background-position: center;
}
```

Property	Definition
background-image	Sets an image as the box's background instead of a solid color
background-size: cover	Stretches the image to completely fill the box without distorting it
background-position	Controls which part of the image is visible/centered

6. Display Property

Definition — display: Every HTML element has a default "display" behavior that decides how it takes up space on the page and whether other elements can sit beside it. This property lets you change that behavior.

Demo tip: Put 3 `<span>` vs 3 `<div>` side by side and toggle display value live.

Value	Definition	Example tag that defaults to it
block	Takes up the FULL width available and always starts on a new line	div, p, h1
inline	Takes up ONLY as much width as its content needs; sits next to other elements on the same line; width/height CANNOT be set	span, a
inline-block	Behaves like inline (sits in a line) BUT width/height CAN be set like block	—
none	Completely hides the element — it takes up no space at all	—

```
.box-inline {
  display: inline-block;
  width: 100px;
  height: 100px;
  background: tomato;
}
```

7. Flexbox Basics

Definition — Flexbox: Flexbox (Flexible Box Layout) is a CSS layout system designed to arrange items in a row or column and easily control their alignment and spacing, without needing tricky floats or manual positioning. You turn a container into a "flex container", and its direct children automatically become "flex items" that line up.

Say to class: "Flexbox = the easiest way to align things in a row or column. This is used in almost every real website."

index.html

```
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
</div>
```

style.css

```
.container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 10px;
}

.item {
  background: #4dabf7;
  padding: 20px;
  color: white;
}
```

Property	Definition
display: flex	Turns this element into a flex container, so its children line up in a row by default
justify-content	Aligns items along the MAIN axis (horizontally, in a row layout) — e.g. space-between spreads them out evenly
align-items	Aligns items along the CROSS axis (vertically, in a row layout) — e.g. center vertically centers them
flex-direction	Changes the direction items flow: row (default, left-to-right) or column (top-to-bottom)
gap	Adds equal spacing between each flex item, without needing margins

8. Positioning

Definition — position: This property controls HOW an element is placed on the page relative to its normal position, its parent, or the whole browser window.

```
.static { position: static; }
.relative { position: relative; top: 10px; }
.absolute { position: absolute; top: 0; right: 0; }
.fixed { position: fixed; bottom: 0; }
```

Value	Definition
static	Default value — element sits in the normal page flow, top/left/right/bottom have no effect
relative	Element stays in normal flow, but you CAN nudge it using top/left/right/bottom — moved relative to its OWN original position
absolute	Element is removed from normal flow and positioned relative to its nearest ANCESTOR that has position:relative (or the page if none found)
fixed	Element is positioned relative to the browser window itself and stays in place even when the page is scrolled

Golden rule to say out loud: "absolute looks for the nearest parent with position:relative to position itself against. Demo this live with a parent-child pair."

9. Mini Practice Project — Profile Card

End class with this: Type it live start to finish, combining everything taught above, and explain each property again briefly as you type it.

index.html

```
<div class="profile-card">
  
  <h2>Riya Sharma</h2>
  <p>Frontend Developer</p>
  <button class="btn">Follow</button>
</div>
```

style.css

```
* { box-sizing: border-box; }

body {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  background: #e9ecef;
  font-family: Arial, sans-serif;
}

.profile-card {
  background: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.15);
  text-align: center;
  width: 220px;
}

.avatar {
  width: 90px;
  height: 90px;
  border-radius: 50%;
  object-fit: cover;
}

.btn {
  background: #3b5bdb;
  color: white;
  border: none;
  padding: 8px 20px;
  border-radius: 6px;
  cursor: pointer;
}

.btn:hover {
  background: #2846b0;
}
```

Definition — :hover and object-fit (new terms used above):

- `:hover` — a "pseudo-class"; the style inside only applies when the mouse is hovering over that element
- `object-fit: cover` — makes an image fill its box completely without stretching/distorting it, cropping if needed
- `cursor: pointer` — changes the mouse cursor to a hand/pointer icon when hovering, signaling it's clickable
- `height: 100vh` — `vh` = "viewport height"; 100vh means 100% of the visible browser window height

Wrap up line for class: "See how every property we learned today — box model, colors, flexbox, border-radius — came together to build one real card. That's how CSS works in real projects."